Functional Paradigm: a hint of Haskell

NAU NORTHERN ARIZONA UNIVERSITY

Haskell: quick start

- Install Haskell locally
  - https://www.haskell.org/downloads/
  - GHC comes with an interactive shell (ghci)

```
ghci> putStrLn(show (5+2))
7
ghci> putStrLn("Hello, World")
Hello, World
```

- Or…
  - https://replit.com/languages/haskell
  - https://onecompiler.com/haskell

```
1  main = putStrLn "Hello, World!"
```

```
> ghc -o main main.hs
[1 of 1] Compiling Main             ( main.hs, main.o )
Linking main ...
> ./main
Hello, World!
>
```

- Haskell is a **strongly typed language**

```
ghci> 5+3
8
ghci> 5+3*4
17
ghci> 5+"3"

<interactive>:6:1: error:
    • No instance for (Num String) arising from the literal ‘5’
    • In the first argument of ‘(+)’, namely ‘5’
      In the expression: 5 + "3"
      In an equation for ‘it’: it = 5 + "3"
ghci>
```

Using functions

- `min` and `max` are two built-in functions (Real, Real) → Real

```
ghci> min 10 14.6
10.0
```

```
ghci> min (max 10 7) (min 28 11)
10
```

```
ghci> min (max 10 (min 12 4)) (min 7 (max 1 2))
2
```

Defining Functions

- Provide the following parts
  - **Name** of the function
  - list of **parameters**
  - The symbol **=**
  - **definition** of the function


```
ghci> squareArea side = side*side
ghci> squareArea 10
100
ghci>
```

Conditionals

- Conditionals are … **FUNCTIONS**!

```
:{
rectangleArea side1 side2 = if side1 == side2
   ;then squareArea side1
   ;else side1*side2
:}
```

```
:{
rectangleArea Double -> Double ->Double
rectangleArea side1 side2 = if side1 == side2
   ;then squareArea side1
   ;else side1*side2
:}
```

- Or…

```
:{
rectangleArea :: Double -> Double -> Double
rectangleArea side1 side2
   | side1 == side2 = squareArea side1
   | otherwise = side1 * side2
:}
```

List Basics

```
ghci> list = [1,2,3,4]
ghci> list
[1,2,3,4]
ghci> list = [1..4]
ghci> list
[1,2,3,4]
ghci> list++[5,6,7]
[1,2,3,4,5,6,7]
ghci> take 2 list
[1,2]
ghci> naturalNumbers [1..]
```

```
ghci> even = [2,4..40]
ghci> even
[2,4,6,8,10,12,14,16,18,20,22,24,26,28,30,32,34,36,38,40]
ghci> multiThree = [3,6..30]
ghci> multiThree
[3,6,9,12,15,18,21,24,27,30]
ghci>
```

Recursive

```
fib :: Integer -> Integer
fib 0 = 0
fib 1 = 1
fib n = fib (n-1) + fib (n-2)
```

```
fib :: Integer -> Integer
fib n
  | n == 0 = 0
  | n == 1 = 1
  | otherwise = fib (n-1) + fib (n-2)
```

See you in our next Lesson!

NAU NORTHERN ARIZONA UNIVERSITY